

Path Smoothing via Footprint-Aware Corner-Transition Optimization for Differential-Drive Mobile Robots

Hai Linh Pham, Tien Cuong Nguyen, Viet Bao Nguyen, Thi Phuong Thao Nguyen, Thi Ly Pham* 

University of Transport and Communications, Vietnam

* Corresponding author. Email: ptyldk@utc.edu.vn

ARTICLE INFO

Received:

Revised:

Accepted:

Published:

KEYWORDS

Differential-drive robot;

Path smoothing;

Quintic Bézier curve;

Robot footprint;

Navigation2; Dynamic programming.

ABSTRACT

Global-planner paths often contain sharp vertices, making continuous turning difficult for differential-drive robots while respecting motion constraints. This paper proposes a footprint-aware corner-transition optimization method, implemented as a post-processing module for the Navigation2 framework of Robot Operating System 2. It replaces neighborhoods of sharp vertices with quintic Bézier curves while ensuring second-order geometric continuity at line–curve junctions. Each transition candidate is validated against curvature, translational velocity, angular velocity, wheel speed, lateral acceleration, and the robot-footprint swept area. Dynamic programming selects a feasible sequence of candidates over the entire path and prevents overlap between transitions at adjacent corners. The method is evaluated using the Jazzy distribution of Robot Operating System 2 and Gazebo Harmonic across three environments and five global planners, yielding 15 inputs for comparison with Raw, Simple, Savitzky–Golay, and Constrained. Results show an average path length of 8.645 m, an integral of squared curvature of 2.887 m^{-1} , and a processing time of 96.0 ms. Relative to Simple and Constrained, this curvature metric is reduced by 75.42% and 90.75%, while path length is reduced by 0.98% and 2.01%, respectively. These results indicate improved geometric path quality without route elongation under the tested conditions.

Doi:

Copyright © JTE. This is an open-access article distributed under the terms and conditions of the [Creative Commons Attribution-NonCommercial 4.0 International License](https://creativecommons.org/licenses/by-nc/4.0/) which permits unrestricted use, distribution, and reproduction in any medium for non-commercial purpose, provided the original work is properly cited.

1. Introduction

Automated Guided Vehicles (AGVs) and Autonomous Mobile Robots (AMRs) are widely used in logistics, smart manufacturing, and intralogistics. Their navigation paths must be safe, kinematically feasible, and compatible with drivetrain limits. In ROS 2 Navigation2 (Nav2), the global planner computes a costmap-based path, while the local controller converts it into velocity commands [1]. Path geometry therefore directly affects tracking accuracy, turning speed, energy demand, and motion continuity.

Grid-search planners typically return polylines. Although convenient for search and collision checking, these paths contain sharp vertices with abrupt tangent changes and discontinuous or undefined curvature. A differential-drive robot cannot track such vertices smoothly at nonzero forward speed while respecting angular and wheel-speed limits. Near a corner, the controller may have to slow sharply, deviate from the reference path, or stop for an in-place rotation, increasing task time and control variation.

Post-planning smoothing improves path executability without changing the global planner. Elastic bands deform a path using attractive and repulsive forces [2]. Geometric primitives and clothoids can provide continuous-curvature motion [3], [4], while analytical methods [5] and Bézier-based methods [6], [7] smooth piecewise-linear paths. Evolutionary curve fitting has also been used for mobile-robot trajectory smoothing [8].

Other approaches formulate smoothing as constrained optimization. Hybrid A* embeds kinematically feasible primitives in search [9], CHOMP uses gradient optimization with obstacle costs [10], and timed elastic-band methods incorporate kinematics, travel time, and nonlinear predictive

control [11], [12], [13]. Distance-field gradients and continuous-state representations have also improved wheeled-robot path quality [14], [15].

However, geometric continuity, low computation, kinematic feasibility, and collision safety are difficult to satisfy simultaneously. Independent corner replacement can create overlapping transitions at nearby corners, while centerline-only collision checks do not represent the area swept by the robot body. Whole-path optimization handles more constraints but is more expensive and may alter the global route substantially. A useful alternative is therefore local smoothing with globally consistent corner decisions, explicit kinematic checks, and full-footprint collision validation.

This paper proposes Path Smoothing via Footprint-Aware Corner-Transition Optimization for Differential-Drive Mobile Robots (PSTMO), a Nav2 post-processing module between the global planner and path-tracking controller. PSTMO replaces only local neighborhoods of sharp vertices, preserving the original route structure. Transition choices are optimized over the complete corner sequence rather than independently, so adjacent transitions remain compatible.

The main contributions of this study are summarized as follows:

- A quintic Bézier corner-transition structure is introduced with boundary conditions that preserve position, tangent direction, and curvature at line–curve junctions.
- Candidate feasibility is checked against differential-drive motion limits and the swept area of the full robot footprint. Dynamic programming selects compatible transitions over the complete path and prevents overlap at adjacent corners.
- Evaluation on 15 map–planner combinations uses path length, integral of squared curvature, and processing time to characterize geometric quality and computational cost.

The remainder of this paper is organized as follows. Section 2 presents the robot model, problem formulation, proposed method, experimental setup, and evaluation metrics. Section 3 presents the results and discussion. Section 4 concludes the paper and outlines future work.

2. Materials and Methods

2.1. Mathematical model and problem formulation

2.1.1. Robot model and path representation

The robot state is $z=(x,y,\psi)$, where (x,y) denotes the geometric-center position and ψ is the heading angle in the map frame. With translational velocity v and angular velocity ω , the kinematic model of a differential-drive robot is given by (1).

$$\dot{x} = v\cos\psi, \dot{y} = v\sin\psi, \dot{\psi} = \omega. \quad (1)$$

The original path from the planner is $P=\{p_0,\dots,p_n\}$, with $p_i=(r_i,\psi_i)$ and $r_i=(x_i,y_i)$. Path length is computed only from planar coordinates according to (2); heading is not included in the distance norm. A point p_i is identified as a corner when the unit incoming vector u and outgoing vector v form the signed heading change $\theta_i=\text{atan2}(u_x v_y - u_y v_x, u \cdot v)$ satisfying $|\theta_i| \geq \theta_{min}$.

$$L(P) = \sum_{i=0}^{n-1} \|r_{i+1} - r_i\|_2. \quad (2)$$

2.1.2. Curvature, continuity, and constraints

For a parametric planar curve $r(t)=(x(t),y(t))$, the signed curvature is defined by (3). Two geometric quantities are used: the maximum absolute curvature κ_{max} and the integral of squared curvature E_κ in (4). E_κ has units of m^{-1} and is a geometric metric, not electrical energy consumption.

$$\kappa(t) = \frac{x'(t)y''(t) - y'(t)x''(t)}{(x'(t)^2 + y'(t)^2)^{3/2}}. \quad (3)$$

$$\kappa_{max} = \max_{s \in [0, L]} |\kappa(s)|, E_{\kappa} = \int_0^L \kappa^2(s) ds. \quad (4)$$

First-order geometric continuity (G^1) requires coincident tangent directions, whereas second-order geometric continuity (G^2) additionally requires curvature continuity. Because a straight segment has $\kappa=0$, a transition connected to a straight segment must satisfy $\kappa=0$ at both endpoints. For active-wheel separation b , the wheel linear velocities and their curvature relationship are given by (5); each candidate must satisfy the limits on translational velocity, angular velocity, individual wheel speed, and lateral acceleration. Figure 1 illustrates the robot model and wheel-velocity convention.

$$v_L = v \left(1 - \frac{b\kappa}{2}\right), v_R = v \left(1 + \frac{b\kappa}{2}\right), \omega = v\kappa. \quad (5)$$

Let F_{body} denote the polygonal robot footprint in the body frame. At pose $z=(x,y,\psi)$, the footprint in the map frame is $F_{map}(z)=[x,y]^T + R(\psi)F_{body}$. The union of footprints at interpolated poses along the path forms the robot-footprint swept area, which must not intersect any obstacle cell. If two consecutive corners share an edge of length L_i , the sum of their trimming distances must also leave a positive non-overlap margin. Figure 2 illustrates the footprint-swept region.

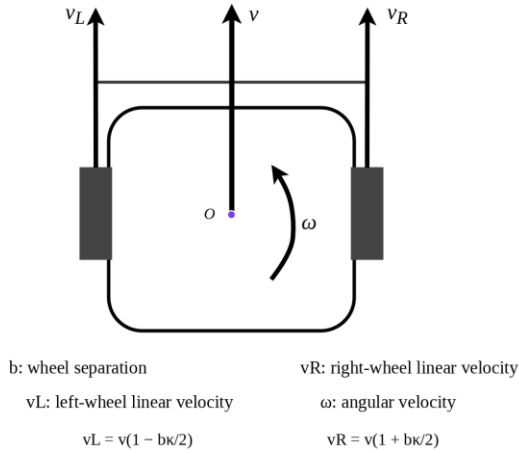


Figure 1. Kinematic model of a differential-drive robot and wheel linear-velocity convention.

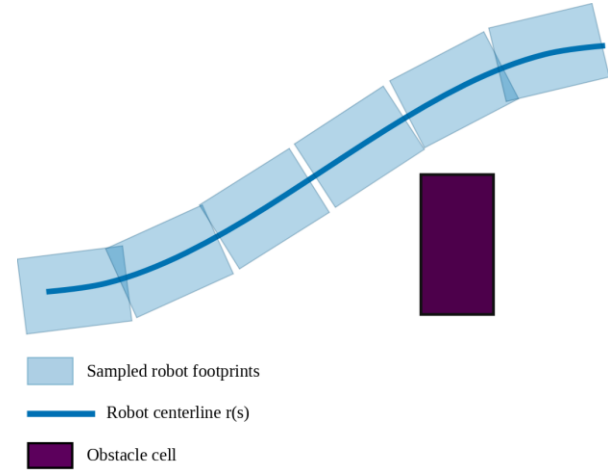


Figure 2. Swept area formed by robot footprints sampled along the centerline; collision is checked over the complete region.

2.1.3. Problem formulation

Inputs are path P , costmap M , footprint F_{body} , and motion limits. The smoothed path P^* must preserve the start, goal, and goal heading; avoid global replanning; and modify only a neighborhood of the input route. Replaced junctions must be G^2 continuous, satisfy (5), remain collision-free under swept-footprint checking, and obey transition non-overlap constraints. Among feasible solutions, PSTMO favors lower E_{κ} , avoids unnecessary path-length increase, and keeps runtime suitable for online post-processing. Small heading changes are retained. Other corners use either a feasible G^2 transition or a safe in-place rotation; if no feasible state exists, the algorithm reports failure.

$$\text{Find } P^* \in \mathcal{F}, \text{ with output quality characterized by the three quantities } [E_{\kappa}(P^*), L(P^*), T_{alg}]. \quad (6)$$

In (6), $\mathcal{F}$ is the set of paths satisfying endpoint, G^2 -continuity, kinematic, swept-footprint, and non-overlap constraints. The quantities $E_{\kappa}(P^*)$, $L(P^*)$, T_{alg} describe smoothness, path length, and processing time but are not combined by fixed weights. PSTMO first rejects candidates outside $\mathcal{F}$ and then compares only feasible candidates, preventing a geometrically smooth but unsafe or infeasible path from being selected.

2.2. Proposed PSTMO method

2.2.1. Method overview

PSTMO receives $P=\{p_0, \dots, p_n\}$ and returns P^* in the same frame. Small zigzags are removed only when the replacement segment remains safe. At each corner, candidates comprise keeping the corner, rotating in place, or inserting a transition. Candidates are validated locally, dynamic programming selects a compatible sequence, and the selected transitions are assembled with the remaining straight segments before a final path-level validation.

A small corner may be retained, a sufficiently open corner may use a transition, and in-place rotation acts as a fallback when translational smoothing is infeasible. This prevents forced rounding in narrow passages. PSTMO still outputs a geometric path; motion-time estimates are used only for screening and selection, not to time-parameterize the output.

2.2.2. Transition construction and parameter selection

Consider a vertex V , with unit vector u on the incoming edge and v on the outgoing edge. The tangency points $A=V-du$ and $B=V+dv$ are defined by the trimming distance d . Let $q=\alpha d$, where $\alpha=q/d$ is a shape ratio. The six control points are arranged as in (7), and the quintic Bézier curve is defined by (8). Figure 3 illustrates the control-point arrangement.

$$\begin{aligned} P_0 &= A, P_1 = A + qu, P_2 = A + 2qu; \\ P_3 &= B - 2qv, P_4 = B - qv, P_5 = B. \end{aligned} \quad (7)$$

$$B(t) = \sum_{i=0}^5 \binom{5}{i} (1-t)^{5-i} t^i P_i, 0 \leq t \leq 1. \quad (8)$$

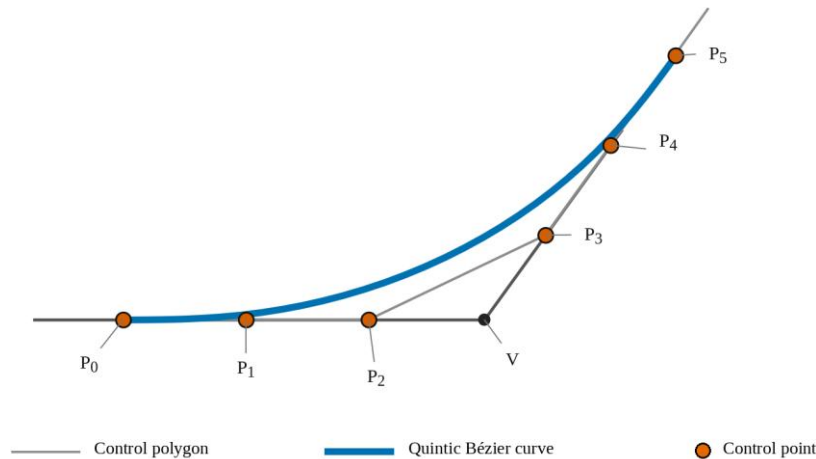


Figure 3. Six control points; collinear, equally spaced endpoint triplets give zero endpoint curvature.

The construction makes $B'(0)$ and $B'(1)$ parallel to the incoming and outgoing edges and gives $B''(0)=B''(1)=0$. Hence, endpoint curvature is zero and the transition has G^2 geometric continuity with the straight segments. Parameter d determines the edge length allocated to the transition, while α controls shape and curvature distribution through $q=\alpha d$. PSTMO derives at most two representative d values—a geometrically preferred value and one compatible with the shared-edge budget—and searches a multilevel α grid for feasible shapes.

Parameter d is bounded by obstacles and neighboring corners because it determines the edge length consumed by the transition. For fixed d , α changes the tangent vector length and curvature distribution. PSTMO therefore retains only (d, α) pairs that pass all mandatory checks rather than imposing one α on every corner.

2.2.3. Transition evaluation and selection

Each transition must have a nondegenerate derivative and finite curvature. Using (5), PSTMO derives angular velocity and wheel speeds from κ and v , then enforces translational-velocity, angular-velocity, wheel-speed, and lateral-acceleration limits. Candidates with unintended curvature reversal, inner-wheel reversal, or no positive feasible velocity are rejected. The complete footprint is then sampled in position and heading along the transition; intersection with any forbidden cell makes the candidate infeasible.

Corners are not independent because two adjacent transitions may consume the same edge. For adjacent corners v_i and v_{i+1} , let d_i and d_{i+1} denote the lengths of their shared edge occupied by the respective candidates, L_i denote the shared-edge length, and m denote the minimum safety margin to be retained between the two transitions. Two consecutive selections are compatible only if the following length-budget constraint is satisfied:

$$d_i + d_{i+1} + m \leq L_i. \quad (9)$$

Figure 4 illustrates the shared-edge budget and the retained margin.

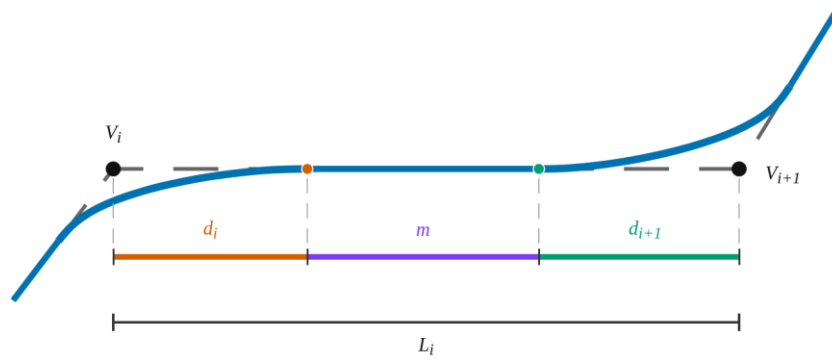


Figure 4. Shared-edge length budget for adjacent corners; margin m prevents transition overlap.

After local feasibility checks, each corner candidate becomes a state. $J_i(k)$ is the local cost of candidate k at corner i , and $F_i(k)$ is the minimum accumulated cost of a feasible sequence ending at that state. For transition states, $J_i(k)$ combines normalized obstacle proximity, peak angular velocity, and E_κ . Only candidates that pass feasibility and runtime screening enter dynamic programming. With $F_1(k)=J_1(k)$, the recurrence for $i=2, \dots, N$ is:

$$F_i(k) = J_i(k) + \min_{j \in C_i(k)} F_{i-1}(j).$$

$C_i(k)$ contains the states j at corner $i-1$ satisfying $d_{i-1}(j)+d_i(k)+m \leq L_{i-1}$; an in-place rotation occupies zero shared-edge length. Each update stores a predecessor pointer. The minimum-cost state at the last corner is selected and the sequence is recovered by backtracking, so compatibility is optimized over the full corner chain instead of repaired after greedy local choices.

If N is the number of corners and each corner has at most K candidates, a current state at each dynamic-programming layer may need to be compared with up to K states from the preceding corner. Therefore, the time complexity of the selection stage is $O(NK^2)$. Objective values are compared only after a candidate has passed all geometric, kinematic, and swept-footprint checks.

2.2.4. Output path construction and validation

The selected transitions are assembled with the remaining straight segments and uniformly resampled. Start position, goal position, and goal heading are preserved. A final independent check verifies finite poses, endpoint preservation, segment feasibility, shared-edge budgets, and collision-free swept-footprint motion over the complete path. Any violation causes PSTMO to report failure.

Path-level validation is necessary because resampling and component junctions can introduce conditions absent from local checks. Acceptance therefore requires finite data, preserved forward route

ordering, and valid footprint samples on the original costmap. Feasibility takes precedence over rounding every corner.

2.3. Experimental setup

Five alternatives are compared: Raw, Simple, Savitzky–Golay, Constrained, and PSTMO. Raw is the planner output and serves as the baseline; each smoother receives exactly the same Raw path. Table 1 summarizes the tested configuration and role of each alternative.

Table 1. Alternatives used in the comparison.

Alternative	Principle/configuration used in the experiment	Role
Raw	Original planner-generated path; no smoothing applied.	Baseline
Simple	Local iterative smoother on path points; up to 1000 iterations, two refinement passes.	Low-cost baseline
Savitzky–Golay	Seven-point polynomial filter with coefficients $[-2, 3, 6, 7, 6, 3, -2]/21$; two refinement passes.	Local-filter baseline
Constrained	Iterative optimization with smoothness and costmap terms; $w_{\text{smooth}}=200000$, $w_{\text{cost}}=0.015$, $w_{\text{curve}}=w_{\text{dist}}=0$.	Constrained baseline
PSTMO	G^2 quintic Bézier transitions, two-wheel limits, swept-footprint checking, and dynamic programming.	Proposed method

Raw is not a smoother. Simple and Savitzky–Golay are low-cost local methods, whereas Constrained performs iterative costmap-aware optimization. Its tested curvature weight is zero, so E_κ is not directly minimized. All configurations are fixed across the three maps and five path sources; the results therefore describe the tested settings rather than the best achievable tuning of each method.

Experiments use Ubuntu 24.04, ROS 2 Jazzy, Nav2, Gazebo Harmonic, and an Intel Core i5-12450HX. The rectangular robot footprint is 0.44×0.34 m. The smoother uses wheel separation $b=0.2548$ m; the Gazebo DiffDrive plugin uses an effective 0.2834 m after wheel–ground and odometry calibration. Table 2 lists the remaining limits and sampling parameters.

Table 2. Main parameters of the experimental system.

Component	Value
Platform	Ubuntu 24.04; ROS 2 Jazzy; Gazebo Harmonic
CPU	Intel Core i5-12450HX
Costmap	0.05 m/cell
Robot footprint	Rectangle, 0.44×0.34 m
Wheel separation	Physical/model: 0.2548 m; effective Gazebo value: 0.2834 m
Velocity limits	$v_{\text{max}}=0.30$ m/s; $ \omega _{\text{max}}=0.80$ rad/s; $ v_{\text{wheel}} _{\text{max}}=0.36$ m/s
Lateral-acceleration limit	0.18 m/s ²
Sampling interval	Validation: 0.02 m; output: 0.05 m

For each environment, NavFn A*, NavFn Dijkstra, Theta*, Smac2D, and SmacHybrid generate five Raw paths. Each smoother is then run independently on those paths, yielding 15 paired map–planner groups. Methods within a group use the same costmap and motion limits. Each combination is run once, so the reported statistics are descriptive and no confidence intervals are claimed.

2.4. Experimental scenarios

Three maps represent different geometries: (i) an open space with a central obstacle, from $(-2.20, -0.60)$ m to $(1.20, -0.60)$ m; (ii) a narrow southwest–northeast diagonal passage, from $(-5.00, -3.00)$

m to (5.00, 3.00) m; and (iii) an intersecting-aisle warehouse, from (−2.00, −2.80) m to (2.00, 2.80) m. All methods produce outputs for all 15 groups. Figures 5–7 show representative Theta* cases.

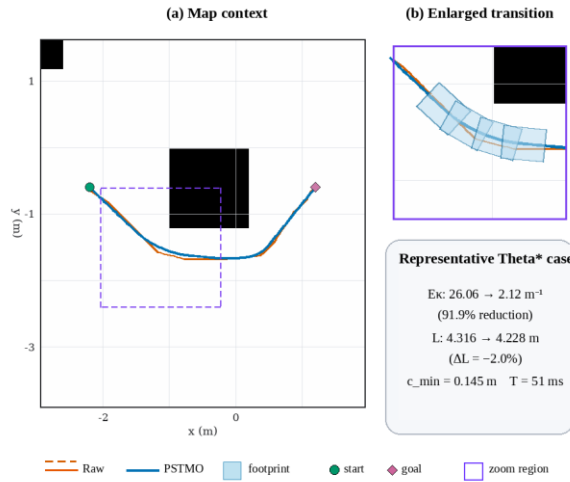


Figure 5. Representative Theta* case in open space: (a) map context; (b) enlarged transition with sampled footprints. Metrics are for this case only.

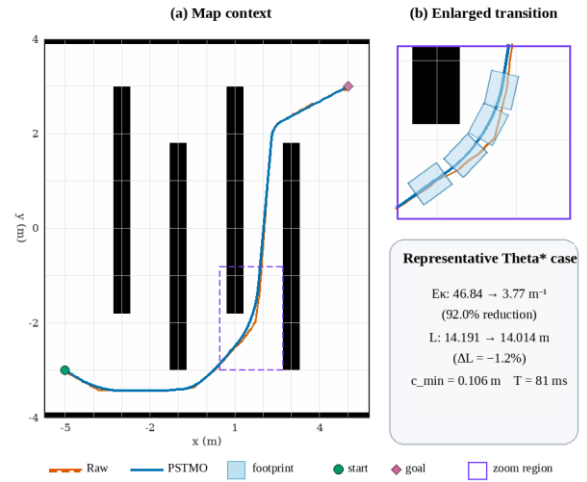


Figure 6. Representative Theta* case in the narrow passage: (a) map context; (b) enlarged transition. The dashed box marks the enlarged region.

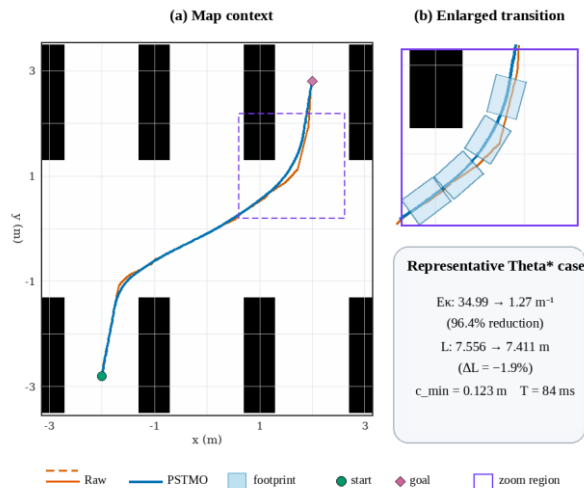


Figure 7. Representative Theta* case in the intersecting-aisle warehouse: (a) map context; (b) enlarged transition. Raw and PSTMO share the same costmap.

2.5. Evaluation metrics

The comparison uses only path length L in (2), integral of squared curvature E_k in (4), and processing time from path reception to return. Raw time measures only the pass-through/copy operation and excludes global planning. Because timer quantization is about 3 ms, sub-millisecond mean values indicate measurements below reliable resolution, not zero cost.

L is the sum of Euclidean distances between consecutive samples. For consistent E_k evaluation across different point densities, every path is arc-length resampled, discrete curvature is squared, and the result is numerically integrated. Means are taken over the 15 paired map-planner cases, ensuring that all smoothers in a group receive the same Raw path.

3. Results and Discussion

Table 3 summarizes the 15 paired cases. PSTMO obtains $E_k=2.887 \text{ m}^{-1}$, reductions of 98.26%, 75.42%, 91.06%, and 90.75% relative to Raw, Simple, Savitzky–Golay, and Constrained. Its mean length is 8.645 m, 0.98%–2.01% shorter than the other methods, so the lower E_k is not obtained by path elongation in this test set. Processing time is 96.0 ms, compared with 1.0 ms for Simple, 0.2 ms for Savitzky–Golay, and 19.0 ms for Constrained.

Table 3. Mean results over 15 map–planner groups.

Method	L (m)	$E_k \text{ (m}^{-1}\text{)}$	T (ms)
Raw	8.818	166.224	4.8
Simple	8.730	11.744	1.0
Savitzky–Golay	8.787	32.286	0.2
Constrained	8.822	31.213	19.0
PSTMO	8.645	2.887	96.0

Raw has high E_k because its discrete heading changes are evaluated after common resampling; E_k should therefore be interpreted as a bending metric under this protocol, not as continuous curvature at an ideal vertex. Simple strongly reduces it, while Savitzky–Golay retains more local variation and Constrained does not directly optimize it in the tested setting. PSTMO forces curvature to zero at transition endpoints. Because the baselines were not tuned for a common objective, the comparison should not be generalized to every possible parameter configuration.

Tables 4–6 present the mean results for the open space, narrow passage, and intersecting-aisle warehouse, respectively.

Table 4. Mean results in the open space.

Method	L (m)	$E_k \text{ (m}^{-1}\text{)}$	T (ms)
Raw	4.334	94.450	1.8
Simple	4.283	12.494	1.8
Savitzky–Golay	4.319	29.529	0.0
Constrained	4.356	17.813	10.2
PSTMO	4.224	1.954	54.0

Table 5. Mean results in the narrow passage.

Method	L (m)	$E_k \text{ (m}^{-1}\text{)}$	T (ms)
Raw	14.409	257.966	9.0
Simple	14.269	8.833	1.2
Savitzky–Golay	14.353	31.678	0.0
Constrained	14.407	57.305	31.2
PSTMO	14.169	4.326	154.2

Table 6. Mean results in the intersecting-aisle warehouse.

Method	L (m)	E_k (m ⁻¹)	T (ms)
Raw	7.711	146.255	3.6
Simple	7.640	13.906	0.0
Savitzky-Golay	7.688	35.651	0.6
Constrained	7.703	18.523	15.6
PSTMO	7.541	2.380	79.8

Tables 4–6 show the lowest mean E_k for PSTMO in all environments: 1.954, 4.326, and 2.380 m⁻¹ for the open space, narrow passage, and intersecting-aisle warehouse. Relative to Simple, the reductions are 84.36%, 51.03%, and 82.89%. The narrow passage has both the smallest relative reduction and the largest PSTMO time, 154.2 ms. This is consistent with adjacent corners sharing the edge budget in (9), although the experiment does not isolate individual causal factors.

PSTMO processing time is 54.0 ms in the open space, 154.2 ms in the narrow passage, and 79.8 ms in the warehouse. The ordering is consistent with the number of corner candidates and compatibility checks rather than path length alone. With only one start–goal pair per map, however, no general runtime model can yet be inferred.

PSTMO is about 1%–2% shorter on average, but it does not find a new global route; corridor selection remains the responsibility of the global planner. Its benefit is to redistribute turning within that route so E_k falls without path elongation, at the cost of additional candidate generation and validation.

The study is limited to three static maps, one start–goal pair per map, five path sources, and one run per combination. Thus, uncertainty and runtime generalization are not yet available, and very small timing values are limited by timer resolution. Collision-free status is binary on the tested costmap and does not quantify physical clearance. E_k is geometric and does not directly measure tracking error, energy, or dynamic loading. The discrete parameter search is not globally optimal in continuous space, and geometric G^2 continuity does not imply zero time-domain jerk.

4. Conclusions

PSTMO is a post-processing method for converting sharp polyline paths into geometry better suited to differential-drive motion. It locally replaces corner neighborhoods with G^2 quintic Bézier transitions while selecting candidates over the full corner sequence. Candidates must satisfy curvature, translational- and angular-velocity, wheel-speed, lateral-acceleration, and swept-footprint collision constraints. Dynamic programming prevents overlap between adjacent transitions, and final path-level validation prioritizes feasibility and safety.

Across 15 combinations of three environments and five global planners, the paired protocol gives every smoother the same Raw input. PSTMO attains the lowest mean E_k in all three environments. Overall, it achieves $L=8.645$ m, $E_k=2.887$ m⁻¹, and $T=96.0$ ms. Relative to Simple and Constrained, E_k decreases by 75.42% and 90.75%, while length decreases by 0.98% and 2.01%. The results indicate improved geometric quality without route elongation, with higher processing cost due to candidate generation, footprint checking, and dynamic-programming selection.

Current evaluation covers geometric quality, path length, and processing time only; tracking error, energy consumption, dynamic loading, and dynamic environments remain outside the scope. The results nevertheless demonstrate PSTMO feasibility under the tested conditions.

Future work will use more maps, start–goal pairs, and repeated trials, and will add tracking error, task time, noise, wheel slip, payload, dynamic obstacles, and real-robot validation. Algorithmic extensions include continuous transition-parameter optimization, footprint-clearance-aware selection, and time parameterization for velocity, acceleration, jerk, and energy analysis.

Acknowledgments

This research received no external funding.

Conflict of Interest

The authors declare no conflict of interest.

Declaration of generative AI and AI-assisted technologies in the writing process

During the preparation of this work, the authors used ChatGPT for English-language editing. After using this tool/service, the authors reviewed and edited the content as needed and take full responsibility for the publication's content.

Data Availability Statement

The data and research code supporting the findings of this study are publicly available at https://github.com/NgvBao02/agv_nav2_research_ws_bao. Additional information is available from the corresponding author upon reasonable request.

REFERENCES

- [1] S. Macenski, F. Martín, R. White, and J. Ginés Clavero, "The Marathon 2: A Navigation System," in Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS), 2020, pp. 2718-2725, doi: 10.1109/IROS45743.2020.9341207.
- [2] S. Quinlan and O. Khatib, "Elastic Bands: Connecting Path Planning and Control," in Proc. IEEE Int. Conf. Robotics and Automation (ICRA), 1993, pp. 802-807.
- [3] S. Fleury, P. Souères, J.-P. Laumond, and R. Chatila, "Primitives for Smoothing Mobile Robot Trajectories," in Proc. IEEE Int. Conf. Robotics and Automation (ICRA), 1993, pp. 832-839.
- [4] M. Brezak and I. Petrović, "Path Smoothing Using Clothoids for Differential Drive Mobile Robots," IFAC Proc. Volumes, vol. 44, no. 1, pp. 1133-1138, 2011, doi: 10.3182/20110828-6-IT-1002.02944.
- [5] K. Yang and S. Sukkarieh, "An Analytical Continuous-Curvature Path-Smoothing Algorithm," IEEE Trans. Robotics, vol. 26, no. 3, pp. 561-568, Jun. 2010, doi: 10.1109/TRO.2010.2042990.
- [6] X. Bu, H. Su, W. Zou, and P. Wang, "Curvature Continuous Path Smoothing Based on Cubic Bezier Curves for Car-Like Vehicles," in Proc. IEEE Int. Conf. Robotics and Biomimetics (ROBIO), 2015, pp. 1453-1458, doi: 10.1109/ROBIO.2015.7418975.
- [7] L. Xu, D. Wang, B. Song, and M. Cao, "Global Smooth Path Planning for Mobile Robots Based on Continuous Bezier Curve," in Proc. Chinese Automation Congress (CAC), 2017, pp. 2081-2085, doi: 10.1109/CAC.2017.8243114.
- [8] V. Parque and T. Miyashita, "Smooth Curve Fitting of Mobile Robot Trajectories Using Differential Evolution," IEEE Access, vol. 8, pp. 82855-82866, 2020, doi: 10.1109/ACCESS.2020.2991003.
- [9] D. Dolgov, S. Thrun, M. Montemerlo, and J. Diebel, "Practical Search Techniques in Path Planning for Autonomous Driving," in Proc. 1st Int. Symp. Search Techniques in Artificial Intelligence and Robotics, 2008.
- [10] N. Ratliff, M. Zucker, J. A. Bagnell, and S. Srinivasa, "CHOMP: Gradient Optimization Techniques for Efficient Motion Planning," in Proc. IEEE Int. Conf. Robotics and Automation (ICRA), 2009, pp. 489-494.
- [11] C. Rösmann, W. Feiten, T. Wösch, F. Hoffmann, and T. Bertram, "Trajectory Modification Considering Dynamic Constraints of Autonomous Robots," in ROBOTIK 2012, 2012, pp. 74-79.
- [12] C. Rösmann, F. Hoffmann, and T. Bertram, "Timed-Elastic-Bands for Time-Optimal Point-to-Point Nonlinear Model Predictive Control," in Proc. European Control Conf. (ECC), 2015, pp. 3352-3357, doi: 10.1109/ECC.2015.7331052.
- [13] C. Rösmann, F. Hoffmann, and T. Bertram, "Kinodynamic Trajectory Optimization and Control for Car-Like Robots," in Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS), 2017, pp. 5681-5686, doi: 10.1109/IROS.2017.8206458.
- [14] E. Heiden, L. Palmieri, S. Koenig, K. O. Arras, and G. S. Sukhatme, "Gradient-Informed Path Smoothing for Wheeled Mobile Robots," in Proc. IEEE Int. Conf. Robotics and Automation (ICRA), 2018, pp. 1710-1717, doi: 10.1109/ICRA.2018.8460818.
- [15] H. Andreasson, J. Saarinen, M. Cirillo, T. Stoyanov, and A. J. Lilienthal, "Fast, Continuous State Path Smoothing to Improve Navigation Accuracy," in Proc. IEEE Int. Conf. Robotics and Automation (ICRA), 2015, pp. 662-669, doi: 10.1109/ICRA.2015.7139250.

Authors' Biography

Hai Linh Pham. Faculty of Electrical-Electronic Engineering, University of Transport and Communications, Hanoi, Vietnam. Research interests include control engineering and automation. Email: linh241632055@lms.utc.edu.vn. ORCID: <https://orcid.org/0009-0000-2277-9922>.

Tien Cuong Nguyen. Faculty of Electrical-Electronic Engineering, University of Transport and Communications, Hanoi, Vietnam. Research interests include control engineering and automation. Email: cuong241632016@lms.utc.edu.vn. ORCID: <https://orcid.org/0009-0000-0141-4561>.

Viet Bao Nguyen. Faculty of Electrical-Electronic Engineering, University of Transport and Communications, Hanoi, Vietnam. Research interests include control engineering and automation. Email: bao241632013@lms.utc.edu.vn. ORCID: <https://orcid.org/0009-0005-4936-1856>.

Thi Phuong Thao Nguyen. Faculty of Electrical-Electronic Engineering, University of Transport and Communications, Hanoi, Vietnam. Research interests include control engineering and automation. Email: thao241632077@lms.utc.edu.vn. ORCID: <https://orcid.org/0009-0003-7285-6975>.

Thi Ly Pham. Faculty of Electrical-Electronic Engineering, University of Transport and Communications, Hanoi, Vietnam. Email: ptyldk@utc.edu.vn. ORCID: <https://orcid.org/0009-0004-1170-6057>. Tel.: +84 987 934 026.